

Day 2 — Python 进阶核心概念（完整版）

一、迭代器（Iterator）

1. 是什么

迭代器是一个可以逐个返回元素的对象。它实现了迭代器协议：`__iter__()` 返回自身，`__next__()` 返回下一个元素，没有更多元素时抛 `StopIteration`。

简单说：能用 `for x in obj:` 遍历的东西，要么本身就是迭代器，要么是可迭代对象（`iterable`）可以转成迭代器。

2. 解决了什么问题

没有迭代器之前，遍历集合你得手动维护索引：

```
i = 0
while i < len(items):
    print(items[i])
    i += 1
```

问题：- 每种数据结构遍历方式不一样（列表用下标，字典用 key，文件读行） - 不是所有东西都有下标（比如生成器、网络流） - 代码重复，容易下标越界

迭代器协议统一了遍历方式：不管底层是什么数据结构，都用 `for x in obj`。

3. 核心理论

迭代器协议包含两个方法：

方法	作用
<code>__iter__(self)</code>	返回迭代器对象自身。让迭代器也可以用在 <code>for</code> 循环中
<code>__next__(self)</code>	返回下一个元素。没有元素时抛 <code>StopIteration</code>

`for x in obj` 本质上做的就是：

```
# for x in obj 等价于：
iter_obj = iter(obj)      # 调 obj.__iter__()
while True:
    try:
        x = next(iter_obj) # 调 iter_obj.__next__()
        # 执行循环体
    except StopIteration:
        break              # 没有更多元素，退出
```

区分两个概念：

概念	定义	例子
可迭代对象 (iterable)	实现了 <code>__iter__</code> ，返回迭代器	列表、字典、字符串、文件
迭代器 (iterator)	实现了 <code>__iter__</code> + <code>__next__</code>	生成器、 <code>iter()</code> 返回值

关键区别：- 可迭代对象每次 `iter()` 都返回新的迭代器，所以可以多次遍历 - 迭代器只遍历一次就耗尽 - 所有迭代器也都是可迭代对象（因为它自己也实现了 `__iter__`）

4. 场景

场景 1：自定义可迭代对象

```
class Countdown:
    """从 n 往下数到 1"""
    def __init__(self, n):
```

```

        self.n = n

    def __iter__(self):
        return CountdownIterator(self.n)

class CountdownIterator:
    def __init__(self, n):
        self.current = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value

for i in Countdown(5):
    print(i)
# 5 4 3 2 1

```

场景 2：用生成器简化（生成器就是迭代器，下一节细讲）

```

class Countdown:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        # 生成器函数自动实现了迭代器协议
        current = self.n
        while current > 0:
            yield current
            current -= 1

for i in Countdown(5):

```

```
print(i)
# 5 4 3 2 1
```

场景 3：惰性遍历大文件（文件对象本身就是迭代器）

```
with open("large_file.log") as f:
    # f 本身就是迭代器，一次读一行，不把整个文件加载到内存
    for line in f:
        process(line)
```

5. 替代方案对比

方式	内存	统一性	灵活性
迭代器协议	低（惰性）	全部统一 <code>for x in obj</code>	自己控制遍历逻辑
手动下标循环	低	不统一（每种结构写法不同）	只能索引访问
一次性加载全部	高（全在内存）	统一	简单但费内存

6. 常见坑

坑 1：迭代器只能遍历一次

```
nums = [1, 2, 3]
it = iter(nums)           # it 是迭代器
print(list(it))           # [1, 2, 3]
print(list(it))           # [] - 耗尽了

# 而列表本身不是迭代器，可多次遍历
print(list(nums))         # [1, 2, 3]
print(list(nums))         # [1, 2, 3] - 每次 iter() 产生新迭代器
```

因为每次 `for x in nums` 都会调 `iter(nums)` 返回一个新的迭代器。

坑 2：在遍历时修改可迭代对象

```
items = [1, 2, 3, 4, 5]
for item in items:
    if item % 2 == 0:
        items.remove(item)    # 遍历时修改！元素会移位跳过
print(items)                  # 结果不一定对

# 解决方案：遍历副本
for item in items[:]:        # items[:] 创建副本
    if item % 2 == 0:
        items.remove(item)
```

坑 3：自定义迭代器忘记 `__iter__`

```
class MyIter:
    def __init__(self, data):
        self.data = data
        self.i = 0

    def __next__(self):
        if self.i >= len(self.data):
            raise StopIteration
        value = self.data[self.i]
        self.i += 1
        return value

it = MyIter([1, 2, 3])
print(next(it))          # 1
print(next(it))          # 2

for x in it:              # TypeError !
    print(x)
```

修复：加 `__iter__`：

```
def __iter__(self):
    return self
```

7. 代码验证

```
class Fibonacci:
    """斐波那契数列迭代器（可指定最大项数）"""
    def __init__(self, max_count):
        self.max_count = max_count
        self.count = 0
        self.a, self.b = 0, 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.max_count:
            raise StopIteration
        value = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1
        return value

for n in Fibonacci(10):
    print(n, end=' ')
# 0 1 1 2 3 5 8 13 21 34
```

迭代器和生成器的关系：生成器是迭代器的一种便捷实现方式。所有生成器都是迭代器，但迭代器不一定是生成器。下一节就讲生成器。

二、闭包（Closure）

1. 是什么

闭包是一个函数 + 它捕获的外部变量的组合体。简单说：函数内部定义的函数，能记住并访问外部函数的变量，即使外部函数已经执行完毕。

2. 解决了什么问题

在需要"一个函数 + 附带状态"的场景下，闭包让你不用写类就能创建带状态的函数。

Python 的函数是一等公民（可以传给别的函数、可以从函数返回），闭包利用这一点实现了轻量级的"函数对象+私有状态"。

如果没有闭包，你只能用：- 全局变量（不安全，容易被意外修改）- 类（太重，有时大材小用）

3. 核心理论

Python 的作用域遵循 LEGB 规则：- Local — 当前函数内 - Enclosing — 外层函数 - Global — 全局 - Built-in — 内置

当内层函数引用外层函数的变量时，Python 会把外层变量"打包"到这个内层函数上，一起返回。只要内层函数还存在，那些被引用的外层变量就不会被垃圾回收。

关键点：被记住的变量是"活的"，不是快照。后续修改外层变量的值，内层函数看到的是修改后的值。

4. 场景

场景 1：计数器

```
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c1 = make_counter()
c2 = make_counter()
print(c1()) # 1
print(c1()) # 2
print(c2()) # 1 - c2 有自己的 count
```

场景 2：函数工厂

```
def make_power(n):  
    def power(x):  
        return x ** n  
    return power  
  
square = make_power(2)  
cube = make_power(3)  
print(square(5)) # 25  
print(cube(5)) # 125
```

场景 3：延迟执行 / 回调

```
def make_greeter(greeting):  
    def greet(name):  
        return f"{greeting}, {name}!"  
    return greet  
  
say_hi = make_greeter("你好")  
say_hello = make_greeter("Hello")  
print(say_hi("小明")) # 你好, 小明!  
print(say_hello("Bob")) # Hello, Bob!
```

5. 替代方案对比

方案	代码量	状态可见性	额外能力
闭包	少	外部不可直接访问（封装性好）	只能有一个入口（函数调用）
类	多	可以通过属性访问	可以有多个方法
全局变量	最少	所有人都能改（不安全）	到处能用（也是缺点）

什么时候用闭包？只需要一个函数、一段简单的状态、不需要多个方法时，闭包比类更简洁。

什么时候用类？状态复杂、需要多个方法操作、需要继承等 OOP 特性时。

6. 常见坑

坑 1：延迟绑定（经典面试题）

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)

print([f() for f in funcs]) # [2, 2, 2], 不是 [0, 1, 2]
```

因为三个 `f` 引用的是同一个 `i`，循环结束时 `i=2`。

修复方案一：默认参数绑定（绑定的是值）

```
for i in range(3):
    def f(x=i): # 用默认参数把当前 i 的值"冻结"进去
        return x
    funcs.append(f)
```

修复方案二：闭包套闭包

```
def make_f(x):
    return lambda: x

for i in range(3):
    funcs.append(make_f(i))
```

坑 2：忘记 **nonlocal**

```
def outer():
    x = 0
    def inner():
        x += 1 # UnboundLocalError! Python 认为 x 是局部变量
        return x
    return inner
```

需要 `nonlocal x` 告诉 Python：这个 `x` 不是本地的，是外层作用域的。

7. 代码验证

```
# 闭包基础
def make_adder(n):
    def add(x):
        return x + n
    return add

add5 = make_adder(5)
print(add5(10))    # 15
print(add5(100))   # 105

# nonlocal 修改外部变量
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c = make_counter()
print(c()) # 1
print(c()) # 2
print(c()) # 3
```

三、装饰器 (Decorator)

1. 是什么

装饰器是一个接受函数作为参数、返回新函数的函数。它让你在不修改原函数代码的前提下，给函数添加额外功能。

`@decorator` 是语法糖，等价于 `func = decorator(func)`。

2. 解决了什么问题

真实项目里经常需要给大量函数加相同的功能：打日志、计耗时、权限校验、缓存、重试逻辑。如果每个函数里都写一遍重复代码，就是严重的 DRY 违反。

装饰器让你把"增强功能"抽出来写一次，然后一行 `@` 就能贴到任意函数上。

3. 核心理论

装饰器的本质是三层结构：

```
def decorator(func):      # 第 1 层：接收原函数
    def wrapper(*args, **kwargs): # 第 2 层：包裹函数，接收原函数的参数
        # 调用前做的事
        result = func(*args, **kwargs) # 调用原函数
        # 调用后做的事
        return result
    return wrapper        # 返回包裹函数
```

`@decorator` 做的事情等价于：

```
def say_hi():
    print("Hi!")

say_hi = decorator(say_hi) # 把 say_hi 替换成 wrapper
```

所以调用 `say_hi()` 时，实际跑的是 `wrapper()`，里面才调了真正的原函数。

4. 场景

场景 1：日志记录

```
import functools

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] 调用 {func.__name__}，参数={args}，{kwargs}")
```

```

        result = func(*args, **kwargs)
        print(f"[LOG] {func.__name__} 返回 {result}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(3, 5)
# [LOG] 调用 add , 参数=(3, 5), {}
# [LOG] add 返回 8

```

场景 2：计时

```

import time
import functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} 耗时 {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

slow_sum(10_000_000)
# slow_sum 耗时 0.3125s

```

场景 3：重试（真实项目的常见需求）

```

import functools
import time

```

```

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"第 {attempt} 次失败: {e}")
                    if attempt == max_attempts:
                        raise
                    time.sleep(delay)
            return None
        return wrapper
    return decorator

@retry(max_attempts=3, delay=0.5)
def unstable_api():
    import random
    if random.random() < 0.7:
        raise ConnectionError("网络不稳定")
    return "成功!"

print(unstable_api())

```

5. 替代方案对比

方案	侵入性	复用性	使用成本
装饰器	零（不改原函数）	高	一行 @
手动在函数内写重复代码	高	低	每个函数都要写
函数包装器模式	低	中	要手动调用 wrapper
回调/钩子	高（改框架）	高	复杂度高

6. 常见坑

坑 1：忘记 `@functools.wraps`

```
def decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@decorator
def my_func():
    """做重要的事情"""
    pass

print(my_func.__name__) # 'wrapper', 不是 'my_func'
print(my_func.__doc__) # None, 不是 '做重要的事情'
```

很多调试工具和框架依赖这些元信息，会出问题。

坑 2：多个装饰器的执行顺序

```
@decorator_a
@decorator_b
def my_func():
    pass

# 等价于：my_func = decorator_a(decorator_b(my_func))
# 执行顺序：先跑 decorator_a 第一次，再跑 decorator_b 第一次
#           然后跑 my_func，再跑 decorator_b 收尾，再跑 decorator_a 收尾
```

两层装饰器像洋葱一样：外层先进入、最后退出。

坑 3：带参数的装饰器需要三层函数

```
# 无参数：两层
@retry
def f(): ...

# 有参数：三层（retry 返回值才是装饰器）
```

```
@retry(max_attempts=3)
def f(): ...
```

7. 代码验证

```
import functools

def validate_positive(func):
    @functools.wraps(func)
    def wrapper(n):
        if n < 0:
            raise ValueError(f"参数必须为正数，收到 {n}")
        return func(n)
    return wrapper

@validate_positive
def sqrt_approx(n):
    return n ** 0.5

print(sqrt_approx(9))    # 3.0
# print(sqrt_approx(-1)) # ValueError
```

四、生成器（Generator）和 yield

1. 是什么

生成器是一个可以暂停和恢复执行的函数。用 `yield` 替代 `return`，函数就变成了生成器。每次调用 `yield`，函数暂停并返回值，下次从暂停处继续。

生成器是迭代器的一种便捷实现。学完迭代器再看生成器，就是"迭代器的自动化版本"。

2. 解决了什么问题

内存问题：当处理大量数据时，一次性全部加载到内存可能不可行。生成器让你"用多少算多少"，一次只生成一个值。

流式处理：数据无限（如实时流、日志流）时，不可能全部存下来。生成器可以不断地产生新数据。

延迟计算：不是每个值都会用到，何必提前算完？生成器惰性求值，用到才算。

3. 核心理论

```
def simple_gen():
    print("开始")
    yield 1
    print("继续")
    yield 2
    print("结束")

g = simple_gen() # 什么都没打印！只是创建了生成器对象
print(next(g))   # 打印"开始"，输出 1
print(next(g))   # 打印"继续"，输出 2
print(next(g))   # 打印"结束"，抛 StopIteration
```

每个 `yield` 像设了一个断点：1. 调用 `next(g)` → 从函数开头或上次暂停处继续执行 2. 遇到 `yield` → 返回值，暂停 3. 下次再调 `next(g)` → 从暂停处继续 4. 没有 `yield` 了 → 抛 `StopIteration`

4. 场景

场景 1：处理大文件（流式读取）

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

# 不管文件多大，一次只读一行到内存
for line in read_large_file("100gb_log.txt"):
    if "ERROR" in line:
        print(line)
```

如果不这么做，`f.readlines()` 会把整个文件读进内存。

场景 2：无限序列

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
print([next(fib) for _ in range(10)])
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# 可以一直取，不会内存溢出
```

场景 3：管道式数据处理

```
def read_ints():
    for i in range(100):
        yield i

def even_only(nums):
    for n in nums:
        if n % 2 == 0:
            yield n

def multiply_by_10(nums):
    for n in nums:
        yield n * 10

# 数据管道 — 每个生成器只处理一步，不创建中间列表
result = multiply_by_10(even_only(read_ints()))
for r in result:
    print(r)
```

5. 替代方案对比

方案	内存	延迟计算	无限序列	消费方式
生成器	低（惰性）	是	支持	只能遍历一次
列表推导式	高（全部在内存）	否	不支持	可多次遍历
生成器表达式	低	是	支持	只能遍历一次

列表 vs 生成器：

```
# 列表 — 全部算完，占内存
squares_list = [x*x for x in range(100_000_000)] # 可能 OOM

# 生成器表达式 — 惰性，几乎不占内存
squares_gen = (x*x for x in range(100_000_000)) # 没问题

# 生成器函数 — 跟表达式等价，但可以更复杂
def squares():
    for x in range(100_000_000):
        yield x*x
```

6. 常见坑

坑 1：生成器只能遍历一次

```
gen = (x for x in range(5))
print(list(gen)) # [0, 1, 2, 3, 4]
print(list(gen)) # [] — 已经耗尽

# 如果以后还要用，生成列表
gen = [x for x in range(5)] # 列表
```

坑 2：生成器只在你要求时才计算

```
def dangerous():
    print("正在执行危险操作...")
```

```

    yield "结果"

g = dangerous() # 没有打印！函数还没执行
# 100 行代码后...
result = next(g) # 才真正执行

```

这既是优点（惰性）也是陷阱（你以为执行了，实际上没有）。

坑 3：**send()** 给 **yield** 传值（高级，了解即可）

```

def accumulator():
    total = 0
    while True:
        value = yield total # yield 表达式可以接收值
        total += value

acc = accumulator()
next(acc) # 必须先启动到第一个 yield
print(acc.send(10)) # 10 - send 给 yield 赋值并推进
print(acc.send(5)) # 15

```

7. 代码验证

```

def batch_reader(data, batch_size=3):
    """将数据分批处理"""
    batch = []
    for item in data:
        batch.append(item)
        if len(batch) == batch_size:
            yield batch
            batch = []
    if batch: # 最后不足一批的
        yield batch

data = [1, 2, 3, 4, 5, 6, 7]
for batch in batch_reader(data):
    print(batch)
# [1, 2, 3]

```

```
# [4, 5, 6]
# [7]
```

五、上下文管理器 (Context Manager)

1. 是什么

上下文管理器是 Python 中定义"进入 → 操作 → 退出"三段式流程的协议。通过 `with` 语句使用，保证"退出"操作一定会执行。

2. 解决了什么问题

大量资源操作有固定模式：申请资源 → 使用 → 释放资源。问题是释放这一步最容易被遗忘或跳过（比如函数中间 `return`、发生异常）。

上下文管理器让 Python 语言本身来保证：不管你怎么退出 **with** 块，`__exit__` 都会跑。

3. 核心理论

`with` 语句的完整执行流程：

```
with EXPR as VAR:
    BLOCK
```

Python 处理为（伪代码）：

```
manager = EXPR          # 计算表达式
VAR = manager.__enter__() # 进入
try:
    BLOCK                # 执行代码块
finally:
    manager.__exit__(...) # 一定退出（不管有没有异常）
```

`__exit__` 的四个参数：- `exc_type` — 异常类型（无异常为 `None`）- `exc_val` — 异常实例 - `exc_tb` — `traceback` 对象 - 返回值：`False` = 不吞异常；`True` = 吞掉异常

4. 场景

场景 1：资源管理

```
# 文件
with open("data.txt") as f:
    data = f.read()
# 自动 f.close(), 哪怕中间有异常

# 数据库连接
class DBConnection:
    def __enter__(self):
        self.conn = sqlite3.connect("db.sqlite")
        return self.conn
    def __exit__(self, *args):
        self.conn.close()

# 锁
import threading
lock = threading.Lock()
with lock:
    # 自动 acquire/release, 不会死锁
    shared_data += 1
```

场景 2：临时状态切换

```
import sys
from io import StringIO

class RedirectStdout:
    def __enter__(self):
        self.old = sys.stdout
        sys.stdout = StringIO()
        return sys.stdout
    def __exit__(self, *args):
        captured = sys.stdout.getvalue()
```

```

        sys.stdout = self.old
        if captured:
            print(f"捕获到的输出：{captured.strip()}")
            return False

with RedirectStdout() as buf:
    print("这段不会显示在终端")
    print("会被捕获到 buf 里")
# 打印：捕获到的输出：这段不会显示在终端\n会被捕获到 buf 里

```

场景 3：事务管理

```

class Transaction:
    def __init__(self, conn):
        self.conn = conn
    def __enter__(self):
        self.conn.execute("BEGIN")
        return self
    def __exit__(self, exc_type, *args):
        if exc_type is None:
            self.conn.execute("COMMIT")
        else:
            self.conn.execute("ROLLBACK")
        return False # 不吞异常

```

5. 替代方案对比

方案	清理保证	可复用	代码噪声
with + 上下文管理器	语言保证	一次定义多处用	低
try/finally	靠人写	每次重写	高
try/except/finally	同上	同上	更高

6. 常见坑

坑 1：__exit__ 返回 **True** 吞异常

```

class SilenceEverything:
    def __exit__(self, *args):
        return True

with SilenceEverything():
    1 / 0 # 不会有 ZeroDivisionError !
print("这行能跑到") # 会被打印，异常被吞了

```

一般情况不要返回 **True**。默认 **False** 是正确的选择。

坑 2： `@contextmanager` 一定要用 **try/finally**

```

from contextlib import contextmanager

@contextmanager
def good_timer():
    start = time.time()
    try:
        yield
    finally:
        print(f"耗时 {time.time() - start:.3f}s")

# 如果 with 块里抛异常，finally 保证取结束时间

```

坑 3： 不是所有"两个操作"都适合做成上下文管理器

上下文管理器适合"做什么之前/之后"的配对操作。纯粹的两个独立步骤不适合。

7. 简写版：`@contextmanager`

```

from contextlib import contextmanager

@contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield # yield 之前 = __enter__, yield 之后 = __exit__
    finally:
        elapsed = time.perf_counter() - start

```

```
        print(f"耗时 {elapsed:.3f}s")

with timer():
    sum(range(10_000_000))
```

8. 代码验证

```
import time

class Timer:
    """计时器上下文管理器"""
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.perf_counter() - self.start
        print(f"耗时 {elapsed:.3f}s")
        return False # 不吞异常

with Timer():
    sum(range(10_000_000))
```

六、五个概念之间的关系

这五个概念不是孤立的，它们可以组合使用：

迭代器 ↔ 生成器：生成器是迭代器的便捷实现。所有生成器都是迭代器。

装饰器 + 上下文管理器

```
def timed_context(cls):
    """装饰一个上下文管理器类，自动计时"""
    import functools
    original_exit = cls.__exit__

    @functools.wraps(original_exit)
```



```

    def timed_exit(self, *args):
        elapsed = time.perf_counter() - self._start
        print(f"{cls.__name__} 耗时 {elapsed:.3f}s")
        return original_exit(self, *args)

    cls.__exit__ = timed_exit
    return cls

@timed_context
class FileHandler:
    def __enter__(self):
        self._start = time.perf_counter()
        return self
    def __exit__(self, *args):
        print("清理资源")
        return False

```

生成器 + 上下文管理器 (@contextmanager)

```

from contextlib import contextmanager

@contextmanager
def open_file(path, mode='r'):
    """结合了生成器和上下文管理器"""
    f = open(path, mode)
    try:
        yield f
    finally:
        f.close()

with open_file("test.txt") as f:
    data = f.read()

```

装饰器 + 生成器

```

def log_yields(func):
    """日志记录生成器每次 yield 的值"""
    import functools
    @functools.wraps(func)

```

```
def wrapper(*args, **kwargs):
    gen = func(*args, **kwargs)
    for value in gen:
        print(f"yield: {value}")
        yield value
    return wrapper

@log_yields
def count_up(n):
    for i in range(n):
        yield i

for x in count_up(3):
    pass
# yield: 0
# yield: 1
# yield: 2
```

总结

概念	一句话	跟其他概念的关系
迭代器	逐个返回元素， <code>__iter__</code> + <code>__next__</code>	生成器是它的简便实现
闭包	函数记住外部变量	装饰器的基础
装饰器	不修改代码给函数加功能	基于闭包实现
生成器	可暂停和恢复的函数 (<code>yield</code>)	是迭代器，也能配合上下文管理器 (<code>@contextmanager</code>)
上下文 管理器	保证进入和退出配对执行	可以用生成器简写 (<code>@contextmanager</code>)

这五个概念共同体现了 Python 的一个核心理念：把重复的模式抽象出来，让语言替你做正确的事。